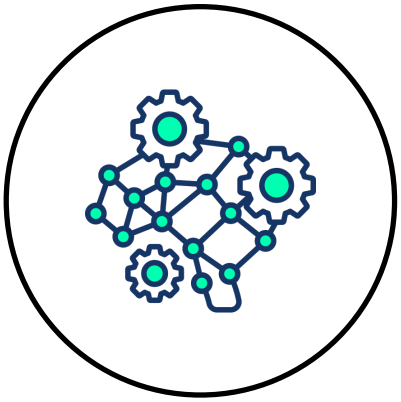
Deep Generative Modeling

University of Manitoba

Assignment 4 (ECE-7650)



Author:

Md. Masud Mazumder

Term: Winter 26

Student ID: 8056333

Instructors:

Ian Jeffrey

Associate Professor

Dept. of Electrical and Computer Engineering

University of Manitoba, Canada

Vahab Khoshdel

Assistant Professor

Dept. of Electrical and Computer Engineering

University of Manitoba, Canada

Code Available: GitHub Repository

Date: April 2, 2026

Declaration

I, Md. Masud Mazumder, attest that the work I am submitting is my own work and that it has not been copied/plagiarized from online or other sources. Any sourced material used for completing this work has been properly cited.

Signature
Md. Masud Mazumder

Use of AI Tools

ChatGPT was used to assist in editing and improving the clarity and readability of the written explanations in this report. All core ideas, experiments, code modifications, and analyses were developed independently by the author. The tool was primarily used to refine the presentation of the text. In a few cases, simple code snippets (e.g., for image visualization) were generated to streamline the workflow. It also assisted with the implementation of Inception Score (IS) and Fréchet Inception Distance (FID) calculations.

Contents

List of Figures

A Problem 1: Comparing DDPM and WGAN-GP

For this problem, the goal is to assess and evaluate the relative strengths and weaknesses of a Denoising Diffusion Probabilistic Model (DDPM) compared to WGAN-GP. For a fair comparison, the same dataset used in Assignment 3, Problem 1, Part A (Fashion-MNIST) is used for both models. The analysis is based on a comparison of the generated images, as well as the computational requirements to train the two models.

A.1 Task 1: Implementing the Diffusion Model

Problem statement: Using the provided Jupyter notebooks, implement a diffusion model for the dataset used in the WGAN-GP assignment.

Solution

Approach: In assignment 3 (WGAN-GP), I have used the Fashion MNIST dataset. Therefore, to solve this task, I have implemented the DDPM model for the Fashion MNIST dataset. The DDPM implementation was adapted from the provided notebook. The provided notebook implements a UNet-based denoising network with a Gaussian diffusion process. Since the original code was designed for 32×32 RGB images, the dataset preparation pipeline was modified to handle the 28×28 single-channel Fashion-MNIST images by resizing them to 32×32 and keeping a single channel.

Implementation: Since the provided notebook was utilized to solve this task, no major changes or augmentations were needed. The key change I have made is for dataset preparation to feed the DDPM architecture. The Fashion-MNIST dataset consists of 60,000 training images of pixel size 28×28 with a single grayscale channel. To prepare the dataset for the DDPM, the images were resized to 32×32 pixels (to match the UNet architecture), rescaled to the range $[-1.0, 1.0]$, and augmented with random horizontal flips. The dataset preparation code is shown in Figure 1.

Generated Images Figure 2 shows images generated by the DDPM after 5 epochs of training. The generated samples show recognizable fashion items, including shirts, pants, shoes, and coats, demonstrating that the model has learned meaningful structure from the Fashion-MNIST distribution.

```python
# Load the Fashion-MNIST dataset
(train_images_raw, train_labels), (test_images, test_labels) = keras.datasets.fashion_mnist.load_data()
print(f"Number of examples: {len(train_images_raw)}")
print(f"Shape of the images in the dataset: {train_images_raw.shape[1:]}")

def augment(img):
    """Flips an image left/right randomly."""
    return tf.image.random_flip_left_right(img)

def resize_and_rescale(img, size):
    """Resize the image to the desired size first and then
    rescale the pixel values in the range [-1.0, 1.0]."""
    img = tf.cast(img, dtype=tf.float32)
    img = tf.image.resize(img, size=size, antialias=True)
    img = img / 127.5 - 1.0
    img = tf.clip_by_value(img, clip_min, clip_max)
    return img

def train_preprocessing(img):
    img = tf.expand_dims(img, axis=-1)
    img = resize_and_rescale(img, size=(img_size, img_size))
    img = augment(img)
    return img

train_ds = (
    tf.data.Dataset.from_tensor_slices(train_images_raw)
    .map(train_preprocessing, num_parallel_calls=tf.data.AUTOTUNE)
    .batch(batch_size, drop_remainder=True)
    .shuffle(batch_size * 2)
    .prefetch(tf.data.AUTOTUNE)
)
```

Figure 1: Dataset preparation code for DDPM on Fashion-MNIST.

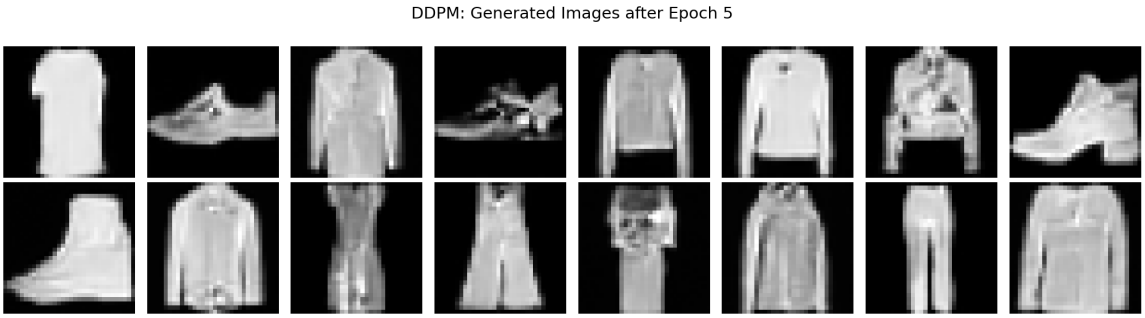


Figure 2: DDPM generated images after Epoch 5.

A.2 Task 2: Training Both Models with Same Hyperparameters

Problem statement: Train both models using the same set of hyperparameters where applicable. Provide a summary and justification for the choices of the relevant model parameters.

Solution

Approach: Since WGAN-GP was already implemented in assignment 3 and DDPM was implemented in task 1 earlier, first, I just placed them into a single notebook (task2.ipynb). Both of these models are using the same Fashion MNIST dataset already. Therefore, without any major changes, I configured the hyperparameters only based on the requirements of this task and trained both models.

Hyperparameter Configuration: To ensure a fair comparison between WGAN-GP and DDPM, the shared hyperparameters were kept identical across both models. The model-specific parameters were chosen based on best practices from the literature. The complete hyperparameter configuration is shown in Figure 3. Also, the shared and model-specific hyperparameters are summarized in Table 1.

```python
# Common hyperparameters and settings
batch_size = 32
img_size = 32
original_img_size = 28
img_channels = 1
clip_min = -1.0
clip_max = 1.0
num_epochs = 5  # Just for the sake of demonstration
learning_rate = 2e-4
beta_1=0.5
beta_2=0.9

# WGAN-specific settings
noise_dim = 128
gp_weight = 10.0
discriminator_extra_steps = 3

# Diffusion model-specific settings
total_timesteps = 1000
norm_groups = 8  # Number of groups used in GroupNormalization layer
first_conv_channels = 64
channel_multiplier = [1, 2, 4, 8]
widths = [first_conv_channels * mult for mult in channel_multiplier]
has_attention = [False, False, True, True]
num_res_blocks = 2  # Number of residual blocks in each stage of the UNet
```

Figure 3: Hyperparameters for both WGAN-GP and DDPM.

Table 1: Summary of hyperparameters used for both models.

Parameter	WGAN-GP	DDPM
Shared Parameters		
Batch size	32	32
Image size	32×32	32×32
Image channels	1	1
Learning rate	2×10^{-4}	2×10^{-4}
β_1 (Adam)	0.5	0.5
β_2 (Adam)	0.9	0.9
Epochs	10	10
WGAN-GP Specific		
Noise dimension	128	–
Gradient penalty weight	10.0	–
Discriminator extra steps	3	–
DDPM Specific		
Total timesteps	–	1000
Normalization groups	–	8
First conv channels	–	64
Channel multiplier	–	[1, 2, 4, 8]
Residual blocks per stage	–	2

Justification of Hyperparameter Choices:

- **Noise dimension (128):** This value is generally set by evaluating multiple values and corresponding outputs with a basic assumption that a more complex dataset would require a higher value. However, to solve this task, I just picked the value from the previous assignment 3 solution, which is also used in the provided notebook.
- **Learning rate (2×10^{-4}):** This is a commonly used learning rate for both GANs and diffusion models, as suggested in the WGAN-GP and DDPM papers. It provides stable training without being too aggressive.
- **Adam parameters ($\beta_1 = 0.5$, $\beta_2 = 0.9$):** These values are standard for GAN training, where a lower β_1 helps prevent the momentum from causing oscillations in the adversarial training process.
- **Gradient penalty weight (10.0):** This is the default value proposed in the original WGAN-GP paper and enforces the Lipschitz constraint on the critic.
- **Discriminator extra steps (3):** Training the critic more frequently than the generator helps ensure accurate Wasserstein distance estimation, as recommended by the WGAN-GP paper. Although the suggested value is 5, I have used 3 to make the computation faster.

- **Noise schedule ($\beta_{\text{start}} = 10^{-4}$, $\beta_{\text{end}} = 0.02$, $T = 1000$):** These are the standard values from the original DDPM paper by Ho et al., providing a gradual noise injection that allows the model to learn effective denoising at each step.
- **Others:** All other parameters are picked directly from the provided notebook.

Dataset Preparation for WGAN-GP: Since the WGAN-GP implementation directly accepts 28×28 images, the images are kept at 28×28 and zero-padded to 32×32 inside the discriminator, whereas for DDPM, images are resized to 32×32 with antialiasing. The WGAN-GP-specific dataset preparation is shown in Figure 4.

```python
# Prepare datasets for WGAN model
train_images = train_images_raw.reshape(-1, original_img_size, original_img_size, img_channels).astype("float32")
train_images = (train_images - 127.5) / 127.5
print(f"WGAN images shape: {train_images.shape}")

wgan_train_ds = (
    tf.data.Dataset.from_tensor_slices(train_images)
    .shuffle(1024)
    .batch(batch_size, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)
```

Figure 4: Additional dataset preparation for the WGAN-GP model.

Generated Images: After training both models for 10 epochs with the same hyperparameters, the generated images are shown in Figures 5 and 6.

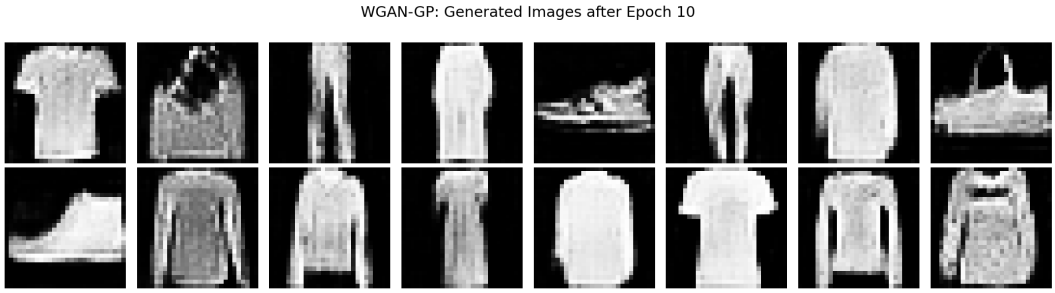


Figure 5: WGAN-GP generated images after Epoch 10.

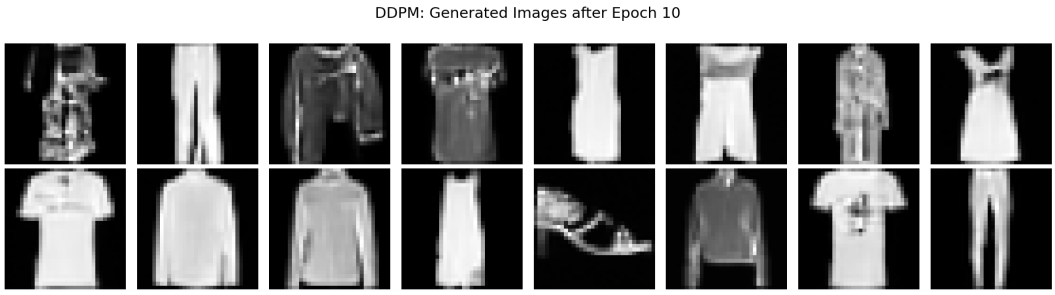


Figure 6: DDPM generated images after Epoch 10.

From visual inspection, the DDPM generates images with noticeably sharper edges and clearer structural details compared to WGAN-GP. The WGAN-GP samples appear noisier and less well-defined after 5 epochs, with some items exhibiting blurred boundaries and artifacts. In contrast, the DDPM samples show well-formed fashion items with clear silhouettes, suggesting that the iterative denoising process produces higher-quality outputs even with limited training.

A.3 Task 3: Quantitative and Qualitative Evaluation

Problem statement: Evaluate both models using qualitative and quantitative metrics. Consider using Inception Score (IS) or Fréchet Inception Distance (FID) for quantitative evaluation and include visual samples from both models for qualitative assessment.

Solution

To quantitatively evaluate the models, two standard metrics were computed: Inception Score (IS) and Fréchet Inception Distance (FID).

1. **Inception Score (IS)** measures the quality and diversity of generated images by evaluating how confidently a pre-trained InceptionV3 classifier can classify the generated images. A higher IS indicates that the model produces sharp, varied images that the classifier assigns to distinct categories with high confidence. Mathematically:

$$\text{IS} = \exp\left(\mathbf{E}_{\mathbf{x}\sim p_g}\left[D_{\text{KL}}\left(p(y|\mathbf{x}) \parallel p(y)\right)\right]\right)$$

2. **Fréchet Inception Distance (FID)** measures the distance between the distribution of real and generated images in the feature space of InceptionV3. A lower FID indicates that the generated images are more similar to real images in terms of both quality and diversity:

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \text{Tr}\left(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2\left(\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_g\right)^{1/2}\right)$$

Implementation: The implementation uses two **InceptionV3** models: one with the classification head for IS computation and one without the top layer (using average pooling) for FID feature extraction. The utility functions for preprocessing and metric calculation are shown in Figure 7.

```python
# Load InceptionV3 models
inception_fid = InceptionV3(include_top=False, pooling="avg", input_shape=(299, 299, 3), weights="imagenet")
inception_is = InceptionV3(include_top=True, input_shape=(299, 299, 3), weights="imagenet")

def preprocess_for_inception(images):
    """Prepare images for InceptionV3: resize to 299x299, convert grayscale to RGB, scale to [-1, 1]."""
    # Ensure float32 and pixel range [0, 255]
    images = tf.cast(images, tf.float32)
    if tf.reduce_max(images) <= 1.0:
        images = images * 255.0

    # Grayscale -> RGB by repeating channels
    if images.shape[-1] == 1:
        images = tf.repeat(images, 3, axis=-1)

    # Resize to 299x299
    images = tf.image.resize(images, (299, 299), method="bicubic", antialias=True)

    # Apply InceptionV3 preprocessing (scales to [-1, 1])
    images = preprocess_input(images)
    return images

def get_inception_features(images, batch_size=32):
    """Extract 2048-d feature vectors from InceptionV3 for FID."""
    images = preprocess_for_inception(images)
    features = inception_fid.predict(images, batch_size=batch_size, verbose=0)
    return features

def get_inception_predictions(images, batch_size=32):
    """Get softmax predictions from InceptionV3 for IS."""
    images = preprocess_for_inception(images)
    preds = inception_is.predict(images, batch_size=batch_size, verbose=0)
    return preds

def calculate_inception_score(images, batch_size=32, splits=10):
    """Calculate Inception Score."""
    preds = get_inception_predictions(images, batch_size)
    scores = []
    n = len(preds)
    for i in range(splits):
        part = preds[i * (n // splits): (i + 1) * (n // splits)]
        p_y = np.mean(part, axis=0, keepdims=True)
        kl_div = part * (np.log(part + 1e-16) - np.log(p_y + 1e-16))
        kl_div = np.mean(np.sum(kl_div, axis=1))
        scores.append(np.exp(kl_div))
    return float(np.mean(scores)), float(np.std(scores))

def calculate_fid(real_images, generated_images, batch_size=32):
    """Calculate Frechet Inception Distance."""
    feat_real = get_inception_features(real_images, batch_size)
    feat_gen = get_inception_features(generated_images, batch_size)

    mu_real, sigma_real = np.mean(feat_real, axis=0), np.cov(feat_real, rowvar=False)
    mu_gen, sigma_gen = np.mean(feat_gen, axis=0), np.cov(feat_gen, rowvar=False)

    diff = mu_real - mu_gen
    covmean, _ = sqrtm(sigma_real @ sigma_gen, disp=False)

    # Numerical stability: remove imaginary components
    if np.iscomplexobj(covmean):
        covmean = covmean.real

    fid = diff @ diff + np.trace(sigma_real + sigma_gen - 2.0 * covmean)
    return float(fid)
```

Figure 7: Utility functions for IS and FID calculation.

The evaluation was performed on 1024 generated images from each model, compared against 1024 real images from the training set. The code for generating samples and computing metrics is shown in Figure 8.

```python
# Generate samples from both models
num_eval_images = 1024
wgan_images = []
ddpm_images = []

for i in tqdm(range(0, num_eval_images, batch_size)):
    n = min(batch_size, num_eval_images - i)
    z = tf.random.normal(shape=(n, noise_dim))
    imgs = wgan.generator(z, training=False)
    imgs = (imgs * 127.5) + 127.5  # scale to [0, 255]
    wgan_images.append(imgs.numpy())
wgan_images = np.concatenate(wgan_images, axis=0)

ddpm_batch = 16  # smaller batch for DDPM since generation is slow
for i in tqdm(range(0, num_eval_images, ddpm_batch)):
    n = min(ddpm_batch, num_eval_images - i)
    imgs = model.generate_images(num_images=n)
    imgs = tf.clip_by_value(imgs * 127.5 + 127.5, 0.0, 255.0)
    ddpm_images.append(imgs.numpy())
ddpm_images = np.concatenate(ddpm_images, axis=0)

# Prepare real images for comparison (same number as generated)
real_images = train_images[:num_eval_images]
real_images = (real_images * 127.5) + 127.5  # scale back to [0, 255]

# Compute Metrics (IS and FID)
is_wgan_mean, is_wgan_std = calculate_inception_score(wgan_images)
is_ddpm_mean, is_ddpm_std = calculate_inception_score(ddpm_images)

fid_wgan = calculate_fid(real_images, wgan_images)
fid_ddpm = calculate_fid(real_images, ddpm_images)

# Print Results
print("\n" + "=" * 55)
print(f"{'Metric':<25} {'WGAN-GP':>12} {'DDPM':>12}")
print("=" * 55)
print(f"{'Inception Score (IS)':<25} {is_wgan_mean:>8.2f}±{is_wgan_std:<4.2f} {is_ddpm_mean:>7.2f}±{is_ddpm_std:<4.2f}")
print(f"{'FID (lower is better)':<25} {fid_wgan:>12.2f} {fid_ddpm:>12.2f}")
print("=" * 55)
```

Figure 8: Code for generating evaluation samples and computing IS and FID.

Quantitative Evaluation Results and Analysis: The quantitative results are summarized in Table 2.

Table 2: Quantitative evaluation results: IS and FID comparison.

Metric	WGAN-GP	DDPM
Inception Score (IS) ↑	3.53 ± 0.23	3.91 ± 0.18
FID (lower is better) ↓	61.07	36.11

The quantitative results clearly favor the DDPM over WGAN-GP on both metrics:

- **Inception Score:** DDPM achieves a higher IS of 3.91 ± 0.18 compared to 3.53 ± 0.23 for WGAN-GP. This indicates that the DDPM generates images that are both sharper and more diverse, as the InceptionV3 classifier assigns them to distinct categories with greater confidence. Additionally, the smaller standard deviation (0.18 vs. 0.23) suggests that DDPM produces more consistently high-quality samples.
- **FID:** DDPM achieves a significantly lower FID of 36.11 compared to 61.07 for WGAN-GP. Since FID measures the distributional distance between real

and generated images in InceptionV3 feature space, the lower value indicates that DDPM-generated images more closely match the statistical properties of the real Fashion-MNIST distribution. The difference of approximately 25 FID points is substantial and demonstrates a clear advantage for the diffusion model.

Qualitative Evaluation Results and Analysis: For qualitative assessment, 16 images from each model are compared side by side in Figure 9.

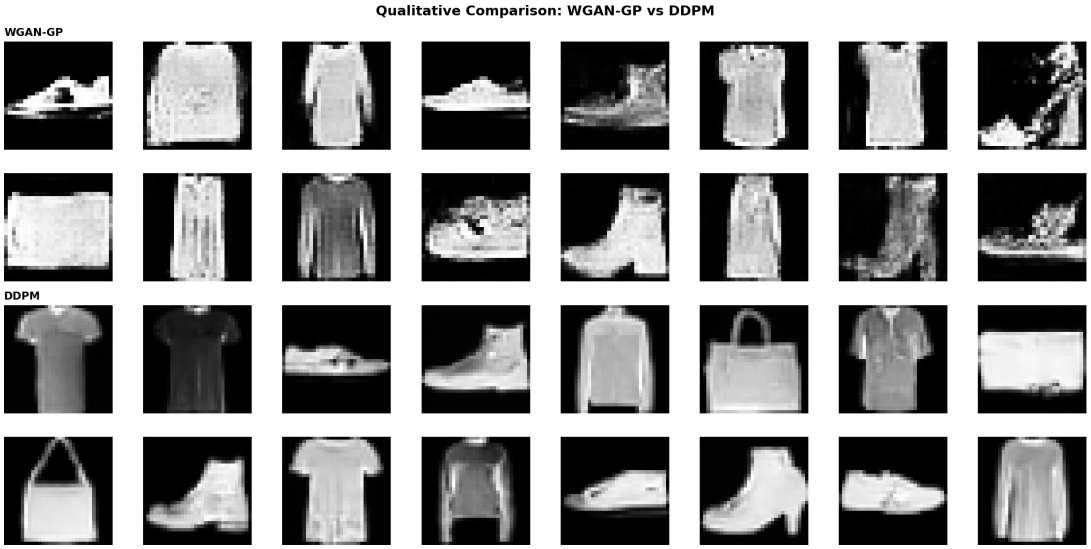


Figure 9: Qualitative comparison of generated images: WGAN-GP (top two rows) vs. DDPM (bottom two rows).

Several observations can be made from the qualitative comparison:

- **Image clarity:** DDPM-generated images exhibit sharper edges and more defined structural details. The outlines of clothing items (shirts, shoes, bags) are cleaner and more distinct compared to WGAN-GP outputs.
- **Diversity:** Both models generate a variety of fashion items across different categories. However, DDPM samples show a wider range of item types and shapes, consistent with its higher Inception Score.
- **Artifacts:** WGAN-GP images contain more noticeable noise and blurring artifacts, particularly around the edges of items and in background regions. DDPM images are smoother and more visually coherent.
- **Structural integrity:** DDPM better preserves the structural properties of fashion items. For example, shoes and bags maintain their characteristic shapes more faithfully in DDPM outputs.

A.4 Task 4: Training Dynamics Comparison

Problem statement: Compare the training dynamics of both models. Provide training curve plots and discuss any significant observations regarding convergence rates, stability, and overall behavior.

Solution

Implementation: To compare the training dynamics, both models were trained for 10 epochs, and the loss values were recorded throughout training. Since the WGAN-GP implementation was already returning the training loss history, for the DDPM, a `loss_tracker` metric was added to the `DiffusionModel` class to properly track the mean squared error loss across batches. The relevant modification is shown in Figure 10.

```python
class DiffusionModel(keras.Model):
    def __init__(self, network, ema_network, timesteps, gdf_util, ema=0.999):
        super().__init__()
        self.network = network
        self.ema_network = ema_network
        self.timesteps = timesteps
        self.gdf_util = gdf_util
        self.ema = ema
        # loss tracker to log the loss values during training
        self.loss_tracker = keras.metrics.Mean(name="loss")

    # Override the metrics property to include the loss tracker
    @property
    def metrics(self):
        return [self.loss_tracker]

    ...

    # Update and return loss values
    self.loss_tracker.update_state(loss)
    return {"loss": self.loss_tracker.result()}
```

Figure 10: Modifications to the DiffusionModel class for loss tracking.

The training curves were plotted using three subplots: WGAN-GP losses, DDPM loss, and a normalized convergence comparison. The plotting code is shown in Figure 11.

```python
# Get epoch numbers for plotting
wgan_epochs = range(1, len(wgan_history.history["d_loss"]) + 1)
ddpm_epochs = range(1, len(ddpm_history.history["loss"]) + 1)

# Plot training losses and convergence comparison
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
axes[0].plot(wgan_epochs, wgan_history.history["d_loss"], label="Discriminator Loss", color="#e74c3c")
axes[0].plot(wgan_epochs, wgan_history.history["g_loss"], label="Generator Loss", color="#2ecc71")
axes[0].set_title("WGAN-GP: Training Losses", fontsize=13)
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(ddpm_epochs, ddpm_history.history["loss"], label="Noise Prediction Loss (MSE)", color="#3498db")
axes[1].set_title("DDPM: Training Loss", fontsize=13)
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Normalize losses for convergence comparison
def normalize(vals):
    vals = np.array(vals, dtype=np.float32)
    return (vals - vals.min()) / (vals.max() - vals.min() + 1e-8)

axes[2].plot(normalize(wgan_history.history["g_loss"]), label="WGAN-GP Generator (normalized)", color="#2ecc71")
axes[2].plot(normalize(wgan_history.history["d_loss"]), label="WGAN-GP Discriminator (normalized)", color="#e74c3c")

ddpm_norm = normalize(ddpm_history.history["loss"])
ddpm_x = np.linspace(0, len(wgan_history.history["g_loss"]) - 1, len(ddpm_norm))
axes[2].plot(ddpm_x, ddpm_norm, label="DDPM Loss (normalized)", color="#3498db", linestyle="--")

# Align epochs for better comparison
axes[2].set_title("Convergence Comparison (Normalized)", fontsize=13)
axes[2].set_xlabel("Epoch (aligned)")
axes[2].set_ylabel("Normalized Loss")
axes[2].legend()
axes[2].grid(True, alpha=0.3)

fig.suptitle("Training Dynamics: WGAN-GP vs DDPM", fontsize=15, fontweight="bold")
plt.tight_layout()
fig.savefig("result/samples/training_dynamics.png", dpi=150, bbox_inches="tight")
plt.show()

# Display key observations from the training dynamics and final metrics
print("\nKey Observations:")
print(f"  WGAN-GP — Final d_loss: {wgan_history.history['d_loss'][-1]:.4f}, g_loss: {wgan_history.history['g_loss'][-1]:.4f}")
print(f"  DDPM    — Final loss:   {ddpm_history.history['loss'][-1]:.4f}")
print(f"  WGAN-GP trained for {len(wgan_history.history['d_loss'])} epochs")
print(f"  DDPM    trained for {len(ddpm_history.history['loss'])} epochs")
```

Figure 11: Code snippet for plotting training dynamics and convergence comparison.

Training Curve Analysis: The training curves for both models are shown in Figure 12.

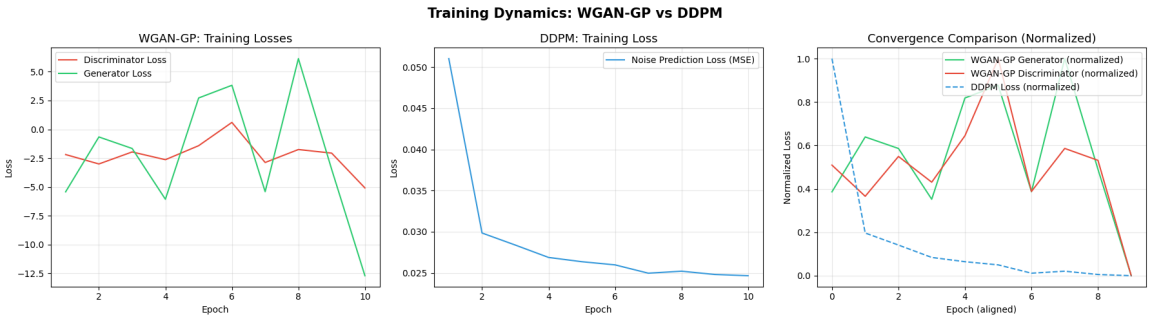


Figure 12: Training dynamics: WGAN-GP vs DDPM. Left: WGAN-GP discriminator and generator losses. Center: DDPM noise prediction loss (MSE). Right: Normalized convergence comparison.

The observations from these training curves are listed below:

1. **WGAN-GP Training Losses (Left Panel):** The WGAN-GP training exhibits significant instability throughout the 10 epochs. The discriminator loss fluctuates between approximately -5 and $+1$, while the generator loss shows even larger oscillations, ranging from approximately -13 to $+6$. The losses do not converge to stable values; instead, they exhibit large swings characteristic of the adversarial training dynamic where the generator and discriminator continuously compete with each other. By epoch 10, the generator loss drops sharply to around -13, which may indicate the generator is finding ways to exploit the critic rather than producing genuinely better samples.

2. **DDPM Training Loss (Center Panel):** In contrast, the DDPM training loss (MSE for noise prediction) shows smooth and monotonic convergence. The loss starts at approximately 0.05 and quickly decreases to about 0.025 within the first few epochs, after which it continues to decrease gradually. This stable convergence is expected because DDPM training is formulated as a straightforward regression problem (predicting the noise added at each timestep) rather than an adversarial minimax optimization.

3. **Convergence Comparison (Right Panel):** The normalized convergence plot provides a direct comparison of the convergence behavior. The DDPM loss (dashed blue line) decreases smoothly and converges near zero, indicating efficient and stable learning. In contrast, the WGAN-GP generator and discriminator losses (green and red solid lines) show large oscillations with no clear trend toward convergence. This comparison highlights a fundamental advantage of diffusion models: their training objective is well-behaved and leads to predictable convergence, whereas GAN training dynamics are inherently unstable due to the adversarial optimization.

Summary of Training Dynamics: The training dynamics comparison reveals several key differences:

Table 3: Training dynamics comparison between WGAN-GP and DDPM.

Aspect	**WGAN-GP**	**DDPM**
Convergence	Oscillatory, unstable	Smooth, monotonic
Loss behavior	Large fluctuations	Steady decrease
Training stability	Sensitive to dynamics	Highly stable
Optimization type	Adversarial minimax	Regression (MSE)

A.5 Discussion: Pros and Cons

Problem statement: Discuss the pros and cons of each model based on your experiments. Consider aspects such as training stability, complexity, quality, and diversity of the generated samples.

Answer

Based on the experimental results, the following comparison can be drawn:

- **DDPM – Advantages**
  - **Superior image quality:** DDPM generates sharper and more structurally coherent images, as evidenced by the lower FID (36.11 vs. 61.07) and higher IS (3.91 vs. 3.53).
  - **Training stability:** The loss function decreases smoothly and monotonically, making training predictable and easy to monitor. No mode collapse or training failure was observed.
  - **Robustness:** The regression-based training objective avoids the instabilities inherent in adversarial training. The model is less sensitive to hyperparameter choices.
  - **Sample diversity:** The stochastic reverse process naturally produces diverse samples without suffering from mode collapse.
- **DDPM – Disadvantages**
  - **Slow generation:** Sampling requires iterating through all $T = 1000$ timesteps sequentially, making generation significantly slower than WGAN-GP (which requires a single forward pass through the generator).
  - **Higher computational cost:** The UNet architecture with attention mechanisms and the large number of denoising steps make DDPM more computationally expensive during both training and inference. It took almost 3x time to be trained than the WGAN-GP.
  - **Model complexity:** The DDPM requires additional components such as the noise schedule, EMA network, and diffusion utilities, increasing implementation complexity.
- **WGAN-GP – Advantages**
  - **Fast generation:** Once trained, generating images requires only a single forward pass through the generator network, making it orders of magnitude faster than DDPM at inference time.
  - **Simpler architecture:** The generator and discriminator are standard convolutional networks that are straightforward to implement and understand.

  - **Lower memory usage:** The WGAN-GP architecture is smaller and requires less GPU memory compared to the UNet used in DDPM.

- **WGAN-GP – Disadvantages**
  - **Training instability:** The adversarial training dynamic leads to oscillating losses and requires careful hyperparameter tuning to avoid mode collapse or training divergence.
  - **Lower image quality:** Generated images contain more noise and blurring artifacts compared to DDPM outputs.
  - **Sensitive to hyperparameters:** As demonstrated in Assignment 3, WGAN-GP training is highly sensitive to the learning rate and other hyperparameters.
  - **Gradient penalty overhead:** The gradient penalty computation adds significant per-step training cost (approximately 4× slower per step compared to standard GAN training, as observed in Assignment 3).

B Discussion Question: Robustness of Diffusion Models over GANs

Question: Assuming that training went well for both models, can you provide some justification arguing for the robustness of diffusion models over GANs? Can you demonstrate a sensitivity to training parameters for WGAN-GP that doesn't exist for the DDPM?

Answer

Theoretical Justification for Robustness: The robustness of diffusion models over GANs can be attributed to fundamental differences in their training objectives:

1. **Non-adversarial training:** DDPM training is formulated as a simple regression problem, where the model learns to predict the noise added at each diffusion step. The loss function is the mean squared error between the true noise and the predicted noise:

$$\mathcal{L}_{\text{DDPM}} = \mathbf{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}} \left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2 \right]$$

   This is a well-behaved convex-like optimization problem with a clear global minimum, unlike the adversarial minimax objective in GANs.

2. **No mode collapse:** GANs are prone to mode collapse, where the generator learns to produce a limited set of outputs that fool the discriminator. Diffusion models avoid this because they learn to reverse a fixed noise-injection process across the entire data distribution, ensuring coverage of all modes.

3. **Stable gradients:** In GAN training, gradient flow depends on the discriminator's ability to provide useful feedback. If the discriminator becomes too strong or too weak, the generator receives vanishing or uninformative gradients. DDPM avoids this issue because the training signal comes directly from the noise prediction error, which provides stable and informative gradients at every training step.

Empirical Evidence (Sensitivity to Learning Rate): From the experiments in Assignment 3 (Task 2), it was demonstrated that WGAN-GP training is highly sensitive to the learning rate. When the learning rate was increased from 10^{-4} to 5×10^{-4} or 10^{-3}, the WGAN-GP loss values exhibited extremely large oscillations and spikes, indicating unstable training. This sensitivity arises because adversarial training requires a delicate balance between the generator and discriminator: a learning rate that is too high causes the Wasserstein distance estimation to become inaccurate, destabilizing the entire training process.

In contrast, the DDPM training remains stable across a wider range of learning rates because the training objective is a standard regression loss. There is no adversarial balance to maintain, and the gradient penalty computation (which introduces

additional sensitivity in WGAN-GP) is not needed. The smooth convergence of the DDPM loss curve in Figure 12 provides clear evidence of this robustness.

Furthermore, the normalized convergence comparison (right panel of Figure 12) shows that the DDPM achieves near-zero normalized loss within a few epochs with smooth convergence, while the WGAN-GP losses continue to oscillate unpredictably even after 10 epochs. This empirically demonstrates the superior robustness and reliability of diffusion models compared to GANs.

C Summary

Overall, the experiments confirm that diffusion models offer significant advantages over GANs in terms of training stability, robustness to hyperparameters, and generation quality. While WGAN-GP maintains an advantage in generation speed, the DDPM's superior image quality (FID: 36.11 vs. 61.07), more diverse outputs (IS: 3.91 vs. 3.53), and predictable training behavior make it a more reliable choice for generative modeling tasks. The fundamental reason for this robustness lies in the non-adversarial training formulation of diffusion models, which eliminates the instabilities inherent in the minimax optimization used by GANs.